

Dialog class and the *QNetworkAccessManager* object can be created in *constructor*, as one object is sufficient to perform all the operations.

The same update operation can be achieved using the GET method and data encoded in the URL as a query string:

```
QUrlQuery querystr;
querystr.addQueryItem("api_key",wRKey);
querystr.addQueryItem("field1","25");
querystr.addQueryItem("field2","72");
querystr.addQueryItem("field3","900");
myurl.setScheme("https");
myurl.setHost("api.thingspeak.com");
myurl.setPath("/update");
myurl.setQuery(querystr);
request.setUrl(myurl);
reply = restclient->get(myurl);
QDebug() << reply->readAll();
```

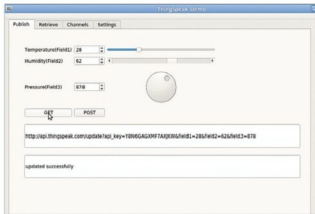


Figure 2: Sending a feed using GET

Alternatively, data encoded in the URL query string format can be used to perform the POST operation using the same URL *api.thingspeak.com/update.json*:

```
//Prepare querystr similar to above steps
request.setHeader(QNetworkRequest::ContentTypeHeader,
    "application/x-www-form-urlencoded");
QByteArray postdata = Qvariant(querystr).toByteArray();
restclient->post(myurl, postdata);
```

Retrieving data from ThingSpeak

To retrieve the feeds from ThingSpeak, we need to perform the GET operation on *http://api.thingspeak.com/channels/xxxxxx/feeds.json* and *api_key=XXX.XX* as the query string. For this purpose, use the following code in the slot connected to the *Retrieve PushButton* click. You can append *results=10* to the query string to limit the number of results to the last 10 feeds.

```
QUrl url;
url.setScheme("http");
```

```
url.setHost("api.thingspeak.com");
url.setPath("/channels/"+CHANNEL+"/feeds.json");
url.setQuery("api_key="+RDKey+"&results=10");
request.setUrl(url);
reply=nwManager->get(request);
```

To process the responses holding all feeds in the JSON format, connect the *finished* signal of *restclient* to a suitable custom slot.

```
QObject::connect(restclient, SIGNAL(finished(QNetworkReply*)),
    this, SLOT(replyFinished(QNetworkReply *)));
```

Parsing JSON data

The returned JSON data has two primary fields with the names *channel* and *feeds*, and the value of *feeds* is an array of all feeds, where each array element consists of fields like *entry_id*, *field1*, *field2*, *field3*, *created_at*, etc.

To traverse all array elements and retrieve the specific fields in each, you can use the following code to parse the JSON data in the *replyFinished* slot:

```
QJsonDocument jsdoc;
jsdoc = QJsonDocument::fromJson(reply->readAll());
QJsonObject jsobj = jsdoc.object();
QJsonArray jsarr = jsobj["feeds"].toArray();
foreach (const QJsonValue &value, jsarr) {
    QJsonObject jsob = value.toObject();
    qDebug() << jsob["entry_id"].toInt();
    qDebug() << jsob["field1"].toString();
    qDebug() << jsob["field2"].toString();
    qDebug() << jsob["field3"].toString();
    qDebug() << jsob["created_at"].toString();
}
reply->deleteLater();
```

Rendering data in tabular form

To display the data in tabular form, create a *TableWidget* through *Table View* with the name *tableFeeds* and add the following code in the slot connected to the *Retrieve PushButton* click.

```
ui->tableFeeds->clearContents();
ui->tableFeeds->setColumnCount(5);
ui->tableFeeds->setRowCount(jsarr.count());
foreach (const QJsonValue &value, jsarr) {
    QJsonObject jsob = value.toObject();
    ui->tableFeeds->setItem(k,0,
        new QTableWidgetItem(jsob["entry_id"].
            toString()));
    ui->tableFeeds->setItem(k,1,
        new QTableWidgetItem(jsob["field1"].toString()));
    ui->tableFeeds->setItem(k,2,
        new QTableWidgetItem(jsob["field2"].toString()));
```